Folhagem & Cia

Carlos Roma | Relatorio e documento da API - PS CITi | 29/10/2026

1. Visao geral, execucao e modelo

A API cadastra plantas, consulta o catalogo e controla as vendas sem deixar o estoque negativo. A implementacao foi integrada ao repositorio de treinamento, mantendo o CRUD de usuarios. A stack e Node.js, Express, TypeScript, Prisma 5.22, PostgreSQL 16 e Docker.

Repositorio: https://github.com/carlosroma-dev/treinamento-crud-citi Branch do trabalho: desafio/carlos-roma.

Para subir do zero, com Git, Node.js 20+, Yarn e Docker Desktop aberto:

```
git clone https://github.com/carlosroma-dev/treinamento-crud-citi.git
cd treinamento-crud-citi
git switch desafio/carlos-roma
Copy-Item .env.example .env
yarn install --frozen-lockfile
docker compose up -d --build
yarn migration
yarn seed
```

A raiz da API fica em http://127.0.0.1:3001. As portas padrao sao 3001 para a API e 5432 para o banco. Se estiverem ocupadas, ajuste SERVER_PORT ou DATABASE_PORT no .env antes de subir. O .env nao vai para o Git. Para ver erros: docker compose logs -f server. O container mantem o reinicio automatico em desenvolvimento.

O id e inteiro automatico. Nome e especie sao textos nao vazios; preco e inteiro positivo em centavos; estoque e inteiro a partir de zero; regaACadaDias e inteiro a partir de um; petFriendly e booleano. Luminosidade aceita SOMBRA, MEIA_SOMBRA ou SOL_PLENO. Os sete campos de negocio sao obrigatorios no POST. createdAt e updatedAt sao automaticos. Inteiros respeitam Int32, ate 2.147.483.647.

Exemplo de cadastro: envie o objeto abaixo sem id, createdAt e updatedAt. A resposta completa, chamada P nos exemplos da pagina seguinte, tem este formato. Id, datas e dados sao ilustrativos:

```
{
  "id": 1,
  "nome": "Planta exemplo A",
  "especie": "Exemplo A",
  "preco": 8990,
  "estoque": 2,
  "luminosidade": "MEIA_SOMBRA",
  "petFriendly": false,
  "regaACadaDias": 3,
  "createdAt": "2026-10-29T12:00:00.000Z",
  "updatedAt": "2026-10-29T12:00:00.000Z"
}
```

Os exemplos sao ficticios; os dados de cuidado e seguranca animal servem apenas para demonstrar os campos da API.

2. Rotas, entradas, respostas e erros

Todas as URLs abaixo usam a base http://127.0.0.1:3001. Corpos de requisicao e resposta usam JSON, com Content-Type: application/json. P representa o objeto completo da pagina anterior; [P] representa uma lista de objetos completos. Os exemplos de alteracao mostram quais campos de P mudam, sem retirar os demais da resposta real.

POST /plantas - Sem parametro. Corpo: os sete campos de negocio do exemplo P. Resposta: 201, objeto P completo. Erros: 400: campo ausente, extra, tipo ou valor invalido. 500: falha inesperada.

GET /plantas - Sem corpo. Filtros opcionais: luminosidade, petFriendly, nome e ordem. Ex.: ?luminosidade=MEIA_SOMBRA&petFriendly=false&nome=exemplo&ordem=preco Resposta: 200, [P]; sem resultado, []. Erros: 400: filtro invalido, desconhecido ou repetido. 500: falha inesperada.

GET /plantas/:id - id inteiro decimal positivo. Ex.: /plantas/1. Sem corpo. Resposta: 200, P. Erros: 400: id invalido. 404: planta inexistente. 500: falha inesperada.

PATCH /plantas/:id - id na URL. Ex.: /plantas/1, corpo {"preco":9500}. Resposta: 200, P com preco 9500 e updatedAt atualizado. Erros: 400: id, corpo ou campo invalido; corpo vazio. 404: planta inexistente. 500: falha inesperada.

DELETE /plantas/:id - id na URL. Ex.: /plantas/1. Sem corpo. Resposta: 204, resposta sem corpo. Erros: 400: id invalido. 404: planta inexistente. 500: falha inesperada.

POST /plantas/:id/venda - Ex.: /plantas/1/venda, corpo {"quantidade":1}. Quantidade inteira positiva. Resposta: 200, P com estoque 1 e updatedAt atualizado. Erros: 400: id ou quantidade invalida, campo extra ou estoque insuficiente. 404: planta inexistente. 500: falha inesperada.

GET /plantas/estoque/valor-total - Sem corpo nem parametros. Soma todas as plantas. Resposta: 200, {"valorTotalEmCentavos":25480,"totalDeVasos":5}. Loja vazia: ambos zero. Erros: 500: falha inesperada.

GET /plantas/esgotadas - Sem corpo. Aceita os mesmos filtros da listagem. Resposta: 200, [P com estoque 0]; se nao houver, []. Erros: 400: filtro invalido, desconhecido ou repetido. 500: falha inesperada.

Nome busca um trecho sem diferenciar maiusculas. Ordem aceita preco ou nome, sempre crescente, com id no desempate. Luminosidade e petFriendly podem ser usados sozinhos ou juntos. No filtro, petFriendly aceita somente os textos true e false. Quando o filtro nao vem, nenhum dos dois valores e excluido.

Os erros das plantas devolvem um objeto com message. Exemplos: {"message":"Planta nao encontrada."} no 404 e {"message":"Estoque insuficiente. Temos 0 vasos."} no 400. JSON malformado, corpo que nao seja objeto, textos vazios, numeros enviados como texto e valores nulos tambem retornam 400.

3. O que foi feito, como e como foi conferido

Entraram as sete rotas obrigatorias, as validacoes, a migration de Planta e os quatro bonus: busca por nome, ordenacao, esgotadas e seed. O trabalho partiu do commit bd7d2f7 do projeto original. Antes de alterar o codigo, passaram sete conferencias HTTP da base. UserController, UserRepository, a conexao existente, o enum Conta e a migration de User foram preservados.

A organizacao segue o modelo do treinamento: routes escolhe o handler, controller valida e responde, repository consulta o Prisma e o banco persiste. A validacao monta um objeto apenas com campos permitidos. No POST, confere presenca e conteudo; no PATCH, confere so o que veio. Isso permite estoque zero e petFriendly false, sem confundir esses valores com campo ausente.

Na venda, o decremento depende de estoque >= quantidade na propria escrita do banco. A operacao e a leitura de retorno ficam na mesma transacao. Se nada mudar, uma consulta distingue planta inexistente de saldo insuficiente. Assim, duas compras nao usam o mesmo ultimo vaso. As rotas fixas foram registradas antes de /plantas/:id.

O total busca apenas preco e estoque e faz a soma com BigInt. A resposta escreve tokens inteiros JSON exatos, inclusive acima de Number.MAX_SAFE_INTEGER. Um cliente JavaScript precisa preservar essa precisao na leitura; JSON.parse convencional pode arredondar valores muito altos no cliente. PATCH vazio, campos desconhecidos e filtros repetidos retornam 400 por escolha de validacao.

A suite integrada passou em 62 de 62 testes, com PostgreSQL real e Prisma 5.22. Sao 55 casos de plantas e sete de regressao de usuarios. Nao ha mock do repository ou do banco. Os testes usam um banco separado e recusam limpeza quando o nome nao contem folhagem_test. Tambem passaram typecheck do codigo e dos testes, build SWC e validacao do schema. A subida foi repetida em volume novo, aplicando as migrations de User e Planta. No Docker com Node 20, os 62 testes passaram. O dist/server.js compilado passou por mais 23 chamadas HTTP, incluindo usuarios, plantas, concorrencia e totais.

A retroverificacao olhou o registro depois da resposta: venda recusada preservou estoque e datas; PATCH so de preco preservou os outros campos; DELETE foi seguido de busca e nova exclusao, com 204, 404 e 404. Duas compras do ultimo vaso deram uma venda e uma recusa. Dez compras para tres vasos deram tres vendas e sete recusas, com saldo zero.

A conta independente foi 8990 x 2 + 2500 x 3 = 25480 centavos, R$ 254,80, com cinco vasos. Depois de vender um vaso de 8990, restaram 16490 centavos, R$ 164,90, e quatro vasos. Os testes conferiram as duas respostas. O seed foi rodado duas vezes: manteve tres plantas, cinco vasos e 25480 centavos, conferidos tambem por soma SQL.

4. Dificuldades, uso de IA e estado da entrega

Uma dificuldade real foi atualizar o Prisma Client dentro do Docker. O volume de node_modules ainda tinha o cliente antigo, que conhecia apenas User. O schema ja tinha Planta, mas a API nao conseguia usar a nova entidade. Gerar o cliente dentro do container e reiniciar a API resolveu. O Dockerfile passou a gerar o cliente antes de iniciar o modo de desenvolvimento. Ao mudar o schema com o ambiente rodando, e preciso conferir a geracao e reiniciar a API para carregar o cliente novo.

O executor dos testes tambem fechou a comunicacao IPC ao usar processos separados; o pool de threads passou com a suite completa. Na revisao, apareceu uma regressao no parser JSON que poderia transformar corpo null em erro 500 nas rotas de usuarios. O modo estrito original foi mantido e um teste novo confirmou o retorno 400. Nenhum caso foi removido para os testes passarem.

Usei Codex, da OpenAI, de forma ampla: implementacao do codigo, criacao dos testes, consultas a documentacao oficial, revisoes independentes e escrita deste documento. Nao foi apenas uma ferramenta para tirar uma duvida pontual. A conferencia comparou o enunciado com o codigo, executou as requisicoes no banco real e refez as contas por outro caminho.

A revisao confirmou pontos que precisam estar claros para explicar a solucao: false e zero sao valores validos; PATCH precisa preservar o que nao foi enviado; a protecao do estoque depende da condicao e do decremento juntos no banco. Ainda preciso fazer minha leitura pessoal do codigo e conseguir explicar cada parte. Testes e revisoes automaticas nao substituem esse entendimento.

O repositorio foi criado como fork de lucascavalcannti/treinamento-crud-citi, mantendo o historico da base. O trabalho esta na branch desafio/carlos-roma. Ha duas orientacoes de entrega: o arquivo recebido pede branch e relato da experiencia; o DESAFIO.md atual pede main e documentacao da API. Este documento cobre o relato e a documentacao. A branch desafio/carlos-roma e a main foram publicadas com a mesma versao final. Nao houve envio pelo canal do processo seletivo nesta etapa.

Para repetir a verificacao, use yarn typecheck e yarn build. Para a suite, configure DATABASE_URL com um banco dedicado contendo folhagem_test no nome, aplique as migrations nesse banco e rode yarn test. O banco principal nao deve ser usado para testes que limpam dados. Estas verificacoes cobrem os casos do desafio; nao representam teste de carga de producao.

Referencias usadas na revisao tecnica:

Prisma ORM: https://www.prisma.io/docs/orm/v6/prisma-client/queries/transactions

PostgreSQL 16, Read Committed: https://www.postgresql.org/docs/16/transaction-iso.html#XACT-READ-COMMITTED